

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN
ARQUITECTURA DEL COMPUTADOR

PRÁCTICA DE LABORATORIO #1

Elaborado por:

Jesús Torres — C.I. 31.022.489

Christian Báez — C.I. 31.473.355

Fecha:

23 de Junio de 2026

Índice

1. ¿Cómo se implementa la recursividad en MIPS32? ¿Qué papel cumple la pila (\$sp)?	2
2. ¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?	2
3. Diferencias entre implementación iterativa y recursiva (Memoria y Registros)	2
4. Diferencias entre ejemplos académicos del libro y un ejercicio completo y operativo	3
5. Tutorial de ejecución paso a paso en MARS	3
6. Justificación del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS	4
7. Análisis y Discusión de los Resultados	4

1. ¿Cómo se implementa la recursividad en MIPS32? ¿Qué papel cumple la pila (\$sp)?

A diferencia de lo que estamos acostumbrados en lenguajes de alto nivel, en MIPS32 la recursividad no es automática; nos toca gestionar todo el proceso manualmente. Cada vez que usamos la instrucción `jal` para llamar a una función, el registro `$ra` cambia, por lo que, si no guardamos la dirección de retorno en la pila, perdemos por completo el camino de vuelta.

Lo que hicimos fue crear un “marco de pila” antes de realizar cada llamada: movimos el puntero `$sp` hacia abajo para hacer el espacio necesario en memoria, guardamos el registro `$ra` y los argumentos correspondientes con la instrucción `sw`, y al terminar la ejecución, recuperamos todos los valores originales con `lw` y subimos el puntero `$sp`. Es básicamente preparar el terreno para no perder el hilo de ejecución del programa.

2. ¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?

Para mitigar este problema en el algoritmo recursivo, vimos que podíamos añadir una validación previa que limite el valor máximo de n permitido según el tamaño del segmento de pila que nos otorga el simulador. Sin embargo, nos dimos cuenta de que la solución definitiva era cambiar el diseño algorítmico: al migrar hacia el enfoque iterativo, eliminamos por completo la dependencia de la pila. Adicionalmente, descubrimos que arquitectónicamente el problema se puede optimizar aún más con una evaluación de máscaras de bits (`andi $v0, $a0, 1`), lo que nos permitió reducir el riesgo de desbordamiento a cero al evaluar directamente el bit menos significativo (LSB) del número entero.

3. Diferencias entre implementación iterativa y recursiva (Memoria y Registros)

Al realizar un análisis comparativo profundo entre ambos enfoques, evidenciamos diferencias críticas en el uso de memoria y registros. La implementación recursiva presenta

una complejidad tanto temporal como espacial de $O(n)$, ya que por cada incremento en el valor de n se genera un nuevo marco de la pila, obligando al sistema a realizar operaciones intensivas de almacenamiento y recuperación de datos (**sw** y **lw**) para resguardar el registro **\$ra** y los argumentos. Esto genera un cuello de botella en el rendimiento y un riesgo inminente de desbordamiento.

Por el contrario, la implementación iterativa tradicional (basada en un bucle de decremento) mantiene una complejidad espacial constante de $O(1)$, dado que no altera el puntero de la pila (**\$sp**) ni requiere salvar la dirección de retorno, eliminando por completo el riesgo de colapso de memoria a costa de mantener un tiempo de ejecución lineal. Finalmente, cabe destacar que si se aplica una optimización puramente aritmética mediante máscaras de bits (**andi**), el algoritmo alcanza el escenario ideal con una complejidad temporal y espacial de $O(1)$, resolviendo el problema en una única instrucción sin tocar la memoria, lo que representa la máxima eficiencia en la arquitectura MIPS32.

4. Diferencias entre ejemplos académicos del libro y un ejercicio completo y operativo

Los ejemplos de los libros suelen ser fragmentos sueltos y muy teóricos. En la práctica, al trabajar directamente con el simulador MARS, notamos que nos faltaban componentes básicos del entorno de desarrollo: las directivas **.data** y **.text** para organizar la estructura de la memoria, y sobre todo, las llamadas al sistema (**syscall**) para poder interactuar de forma real con la consola. Además, tuvimos que aprender a colocar explícitamente el código de finalización del programa (**li \$v0, 10; syscall**), porque de lo contrario, el simulador seguía ejecutando secuencialmente la basura presente en la memoria.

5. Tutorial de ejecución paso a paso en MARS

Para probar y verificar el comportamiento del código estructurado, seguimos la siguiente secuencia: primero abrimos el archivo en el entorno de MARS y lo ensamblamos presionando la tecla **F3**. Una vez posicionados en la pestaña “Execute”, la mejor estrategia para auditar el flujo es avanzar paso a paso presionando **F7**.

Es clave abrir la pestaña “Run I/O” para ingresar el número entero requerido, y mantenerse muy pendientes del banco de registros: al observar directamente cómo el valor de `$sp` bajaba y subía en cada llamada recursiva, logramos entender realmente qué estaba sucediendo en la memoria del procesador.

6. Justificación del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS

Después de hacer todas las pruebas pertinentes, está claro que el enfoque iterativo es el único que deberíamos usar en un entorno real. La recursividad se ve elegante matemáticamente, pero en el lenguaje ensamblador MIPS32 la gestión de la pila lo hace un proceso muy denso, complejo y propenso a errores humanos de asignación. Para un problema tan sencillo como evaluar la paridad, la eficiencia de un bucle simple o una operación lógica de bits no tiene competencia.

Además, mientras que en la recursión dependemos totalmente de la profundidad física de la pila y del tamaño de los datos de entrada, la versión iterativa nos da un control absoluto sobre el flujo de ejecución, permitiendo depurar más fácilmente cualquier error lógico que aparezca durante el desarrollo.

7. Análisis y Discusión de los Resultados

Los resultados obtenidos en el laboratorio fueron reveladores. Con números pequeños, ambos enfoques iban perfecto, pero al probar con un valor de $n = 80000$, la función recursiva simplemente no aguantó. El simulador se congeló por completo debido a que la pila se llenó. Esta experiencia nos dejó claro que la recursión tiene un costo oculto muy alto en bajo nivel. En sistemas embebidos, donde la memoria de almacenamiento y ejecución es sumamente limitada, esta diferencia entre una estructura iterativa limpia y una recursiva pesada es el factor determinante que define si un sistema es estable o si se cae. Definitivamente, este ejercicio nos ayudó a aterrizar conceptos que a veces parecen muy abstractos en el aula. Entender cómo se comporta realmente el procesador y la memoria ante nuestras instrucciones nos da una perspectiva mucho más crítica sobre cómo debemos escribir código de bajo nivel en el futuro, priorizando siempre la robustez sobre la elegancia excesiva.